

String Processing Problems:

Title: String Reversing Problem.

Outline:

Write an assembly language program to implement a string reversing program that will read a string (a line of characters of letters, digits, punctuation symbols, and others) as input and then make the reverse form of the string as output. You have to solve this problem in two ways such as (i) using array (ii) using stack. Your program must contain necessary messages for formatted input and output.

Task: Implementation using Microsoft Macro-Assembler (MASM)

Mode of Evaluation: Experiment + Viva

Percentage of Weight: 5%

Algorithm

Step-1: Begin

Step-2: Declare a SMALL memory MODEL

Step-3: Declare a 100H size STACK

Step-4: Declare an input message

Step-5: Declare an output message

Step-6: Declare X as a 100 bytes array

Step-7: Initialize Data Segment

Step-8: Display input message

Step-9: Move 0 to SI

Step-10: Move 1 to AH

Step-11: Call interrupt 21H

Step-12: Compare AL to 0DH

Step-13: If AL equal to 0DH Then go to Step-17

Step-14: Move AL to X[SI]

Step-15: Increment SI by 1

Step-16: Go to Step-11

Step-17: Decrement SI by 1

Step-18: Display output message

Step-19: Move 2 to AH

Step-20: Move X[SI] to DL

Step-21: Call interrupt 21H

Step-22: Decrement SI by 1

Step-23: Compare SI to 0000H

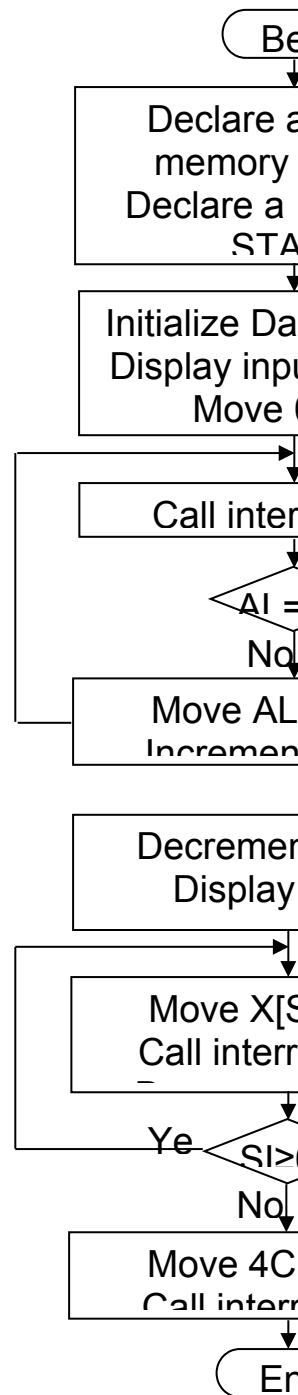
Step-24: If SI is greater than or equal to 0000H Then go to Step-20

Step-25: Move 4CH to AH

Step-26: Call interrupt 21H

Step-27: End

Flowchart:



Source Code:

TITLE : A Program for Reversing a String

```
.MODEL SMALL
.STACK 100H

.DATA
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'Enter a string:',CR,LF, '$'
    MSG2 DB CR,LF,'In reverse it is:',CR,LF, '$'
    X DB 100 DUP(?)

.CODE

MAIN PROC
MOV AX,@DATA
MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV SI,0
    MOV AH,1

NEXT1:
    INT 21H
    CMP AL,0DH
    JE EXIT
    MOV X[SI],AL
    INC SI
    JMP NEXT1

EXIT:
    DEC SI
    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV AH,2
NEXT2:
    MOV DL,X[SI]
    INT 21H
    DEC SI
    JGE NEXT2

    MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```

Sample

Input:

Enter a String:The Quick Brown Fox

Output:

In reverse it is:xoF nworB kciuQ ehT

Title: Case Conversion Problem.

Outline::

Write an assembly language program to implement a case conversion program that will read a string (a line of characters of letters, digits, punctuation symbols, and others) as input and then convert the letters into its opposite case as output. Here, the characters which are not belonged to letters will remain be unchanged. Your program must contain necessary messages for formatted input and output.

Task: Implementation using Microsoft Macro-Assembler (MASM)

Mode of Evaluation: Experiment + Viva

Percentage of Weight: 10%

Algorithm

Step-1: Begin

Step-2: Declare a SMALL memory MODEL

Step-3: Declare a 100H size STACK

Step-4: Declare an input message

Step-5: Declare an output message

Step-6: Declare X as a 100 bytes array

Step-7: Initialize Data Segment

Step-8: Display input message

Step-9: Move 0 to SI

Step-10: Move 1 to AH

Step-11: Call interrupt 21H

Step-12: Compare AL to 0DH

Step-13: If AL equal to 0DH Then go to Step-28

Step-14: Compare AL to 'A'

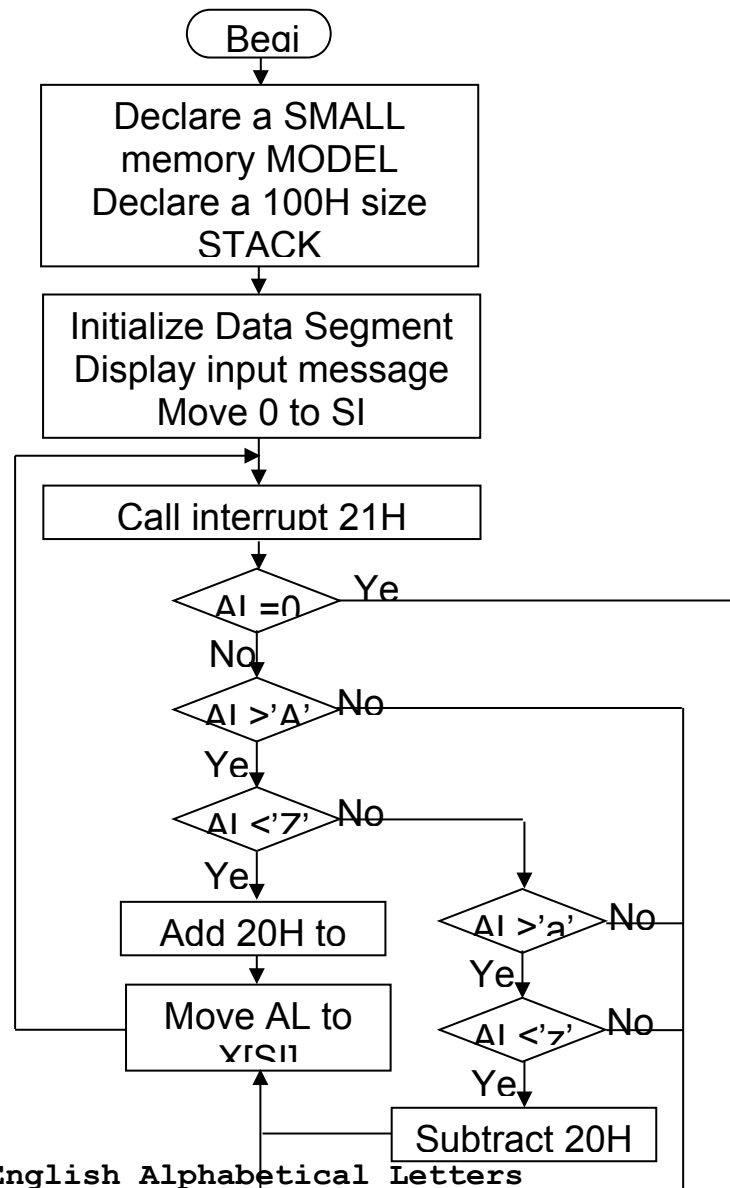
Step-15: If AL is less than 'A' Then go to Step-19

Step-16: Compare AL to 'Z'

Step-17: If AL is greater than 'Z' Then go to Step-22

Step-18: Add 20H to AL
Step-19: Move AL to X[SI]
Step-20: Increment SI by 1
Step-21: Go to Step-11
Step-22: Compare AL to 'a'
Step-23: If AL is less than 'a' Then go to Step-19
Step-24: Compare AL to 'z'
Step-25: If AL is greater than 'z' Then go to Step-19
Step-26: Subtract 20H from AL
Step-27: Go to Step-19
Step-28: Display output message
Step-29: Move 0 to DI
Step-30: Move 2 to AH
Step-31: Move X[DI] to DL
Step-32: Call interrupt 21H
Step-33: Increment DI by 1
Step-34: Compare DI to SI
Step-35: If DI not equal to SI go to Step-31
Step-36: Move 4CH to AH
Step-37: Call interrupt 21H
Step-38: End

Flowchart:



Source Code:

TITLE : A Program for Case Conversion of English Alphabetical Letters

```
.MODEL SMALL
.STACK 100H

.DATA
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'Enter a string:',CR,LF, '$'
    MSG2 DB CR,LF,'In opposite case it is:',CR,LF, '$'
    X DB 100 DUP(?)

.CODE

MAIN PROC
MOV AX,@DATA
```

```
MOV DS,AX
```

```
LEA DX,MSG1  
MOV AH,9  
INT 21H
```

```
MOV SI,0  
MOV AH,1
```

```
NEXT1:  
INT 21H  
CMP AL,0DH  
JE EXIT  
CMP AL,'A'  
JL SKIP2  
CMP AL,'Z'  
JG SKIP1  
ADD AL,20H  
JMP SKIP2
```

```
SKIP1:  
CMP AL,'a'  
JL SKIP2  
CMP AL,'z'  
JG SKIP2  
SUB AL,20H
```

```
SKIP2:  
MOV X[SI],AL  
INC SI  
JMP NEXT1
```

```
EXIT:  
LEA DX,MSG2  
MOV AH,9  
INT 21H
```

```
MOV AH,2  
MOV DI,0
```

```
NEXT2:  
MOV DL,X[DI]  
INT 21H  
INC DI  
CMP DI,SI  
JNE NEXT2
```

```
MOV AH,4CH  
INT 21H
```

```
MAIN ENDP  
END MAIN
```

Sample

Input:

Enter a string:The Quick Brown Fox 123 *#

Output:

In opposite case it is:tHE qUICK bROWN fOX 123 *#

Title: String Counting Problem.

Outline:

Write an assembly language program to implement a counting program that will read a string (a line of characters of letters, digits, punctuation symbols, and others) as input and then count the number of Vowels, Consonants, Digits, and Spaces in the string as output. Your program must contain necessary messages for formatted input and output.

Task: Implementation using Microsoft Macro-Assembler (MASM)

Mode of Evaluation: Experiment + Viva

Percentage of Weight: 15%

Algorithm

Step-1: Begin

Step-2: Declare a SMALL memory MODEL

Step-3: Declare a 100H size STACK

Step-4: Declare an input message

Step-5: Declare an output message for vowels

Step-6: Declare an output message for consonants

Step-7: Declare an output message for digits

Step-8: Declare an output message for spaces

Step-9: Declare a byte variable VOWELS and set to 0

Step-10: Declare a byte variable CONSONANTS and set to 0

Step-11: Declare a byte variable SPACES and set to 0

Step-12: Declare a byte variable DIGITS and set to 0

Step-13: Initialize Data Segment

Step-14: Display input message

Step-15: Move 1 to AH

Step-16: Call interrupt 21H

Step-17: Compare AL to 0DH

Step-18: If AL equal to 0DH Then go to Step-62

Step-19: Compare AL to space character

Step-20: If AL not equal to space character Then go to Step-23

Step-21: Add 1 to SPACES

Step-22: Go to Step-16

Step-23: Compare AL to '0'

Step-24: If AL is less than '0' Then go to Step-16

Step-25: Compare AL to '9'

Step-26: If AL is greater than '9' Then go to Step-29

Step-27: Add 1 to DIGITS

Step-28: Go to Step-16

Step-29: Compare AL to 'A'

Step-30: If AL is less than 'A' Then go to Step-16

Step-31: Compare AL to 'Z'

Step-32: If AL is greater than 'Z' Then go to Step-47

Step-33: Compare AL to 'A'

Step-34: If AL equal to 'A' Then go to Step-45

Step-35: Compare AL to 'E'

Step-36: If AL equal to 'E' Then go to Step-45

Step-37: Compare AL to 'I'

Step-38: If AL equal to 'I' Then go to Step-45

Step-39: Compare AL to 'O'

Step-40: If AL equal to 'O' Then go to Step-45

Step-41: Compare AL to 'U'

Step-42: If AL equal to 'U' Then go to Step-45

Step-43: Add 1 to CONSONANTS

Step-44: Go to Step-16

Step-45: Add 1 to VOWELS

Step-46: Go to Step-16

Step-47: Compare AL to 'a'

Step-48: If AL is less than 'a' Then go to Step-16

Step-49: Compare AL to 'z'

Step-50: If AL is greater than 'z' Then go to Step-16

Step-51: Compare AL to 'a'

Step-52: If AL equal to 'a' Then go to Step-45

Step-53: Compare AL to 'e'

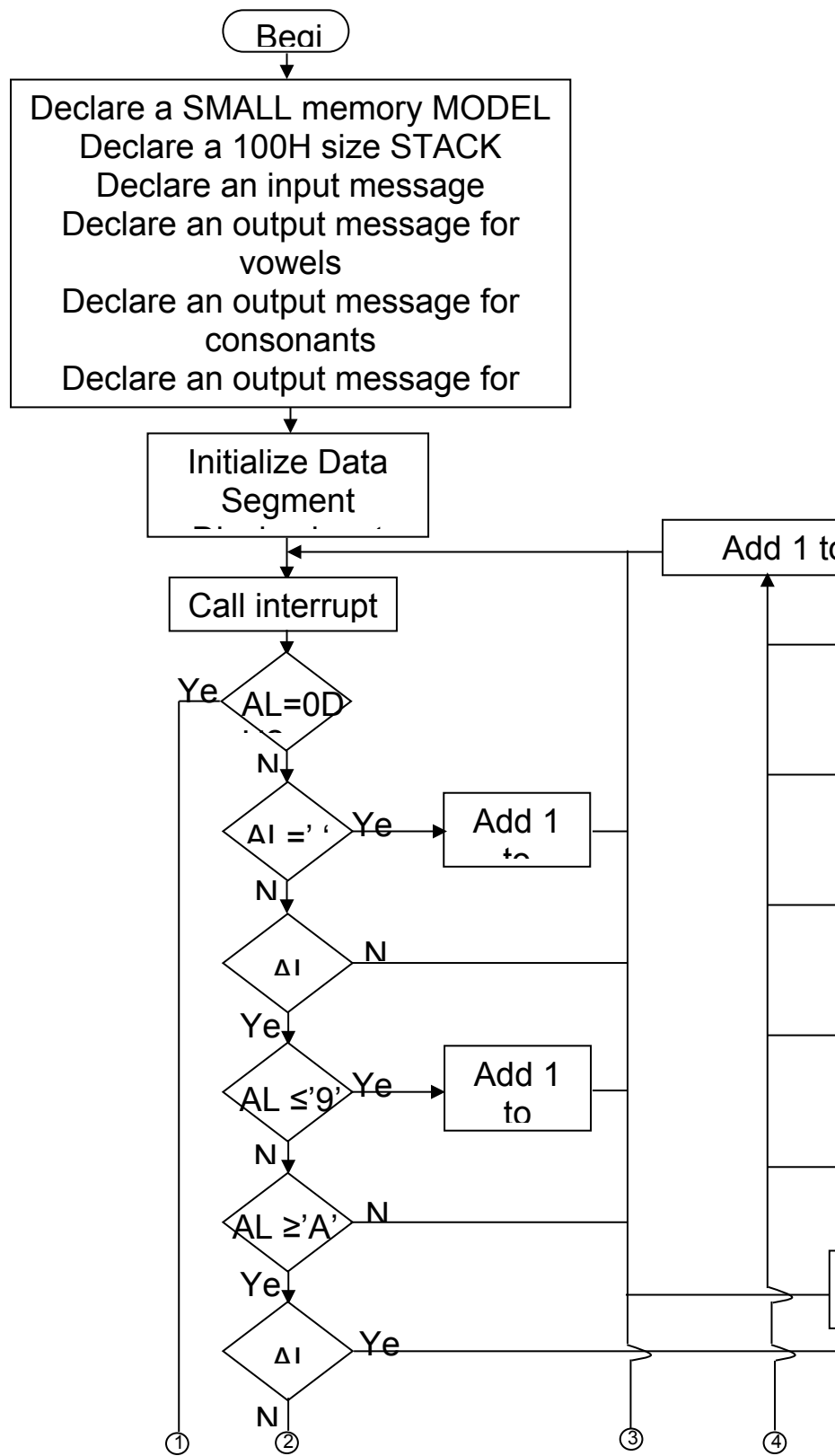
Step-54: If AL equal to 'e' Then go to Step-45

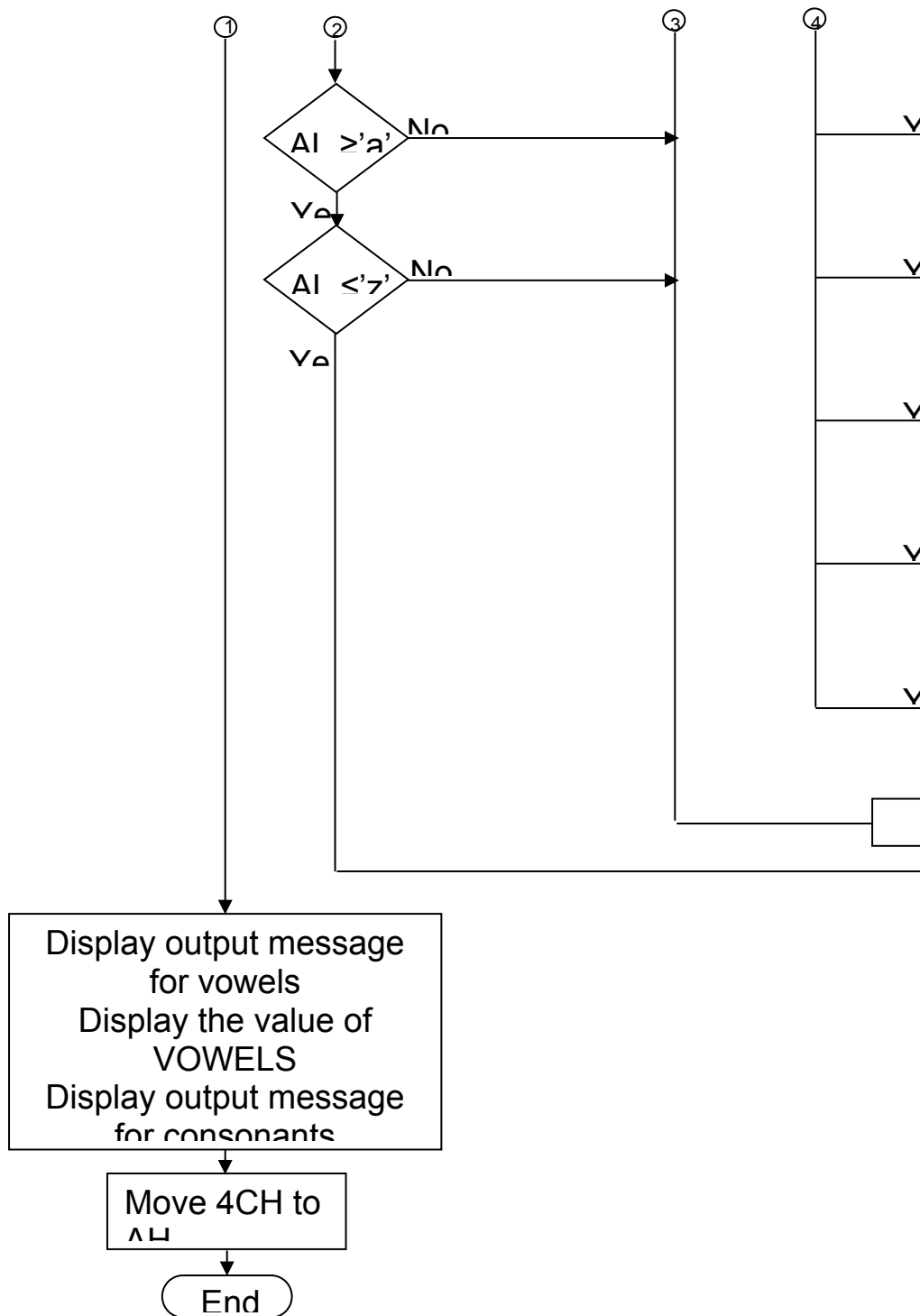
Step-55: Compare AL to 'i'

Step-56: If AL equal to 'i' Then go to Step-45

Step-57: Compare AL to 'o'
Step-58: If AL equal to 'o' Then go to Step-45
Step-59: Compare AL to 'u'
Step-60: If AL equal to 'u' Then go to Step-45
Step-61: Go to Step-43
Step-62: Display output message for vowels
Step-63: Display the value of VOWELS
Step-64: Display output message for consonants
Step-65: Display the value of CONSONANTS
Step-66: Display output message for digits
Step-67: Display the value of DIGITS
Step-68: Display output message for spaces
Step-69: Display the value of SPACES
Step-70: Move 4CH to AH
Step-71: Call interrupt 21H
Step-72: End

Flowchart:





Source Code:

TITLE : A Program for Counting Vowels, Consonants, Digits, and Spaces in a String where each count<10

```
.MODEL SMALL
.STACK 100H

.DATA
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'Enter a string:',CR,LF, '$'
    MSG2 DB CR,LF,'Vowels: $'
    MSG3 DB CR,LF,'Consonants: $'
    MSG4 DB CR,LF,'Digits: $'
    MSG5 DB CR,LF,'Spaces: $'

    VOWELS DB 0
    CONSONANTS DB 0
    DIGITS DB 0
    SPACES DB 0

.CODE

MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV VOWELS,0
    MOV CONSONANTS,0
    MOV DIGITS,0
    MOV SPACES,0

    MOV AH,1

NEXT:

    INT 21H
    CMP AL,0DH
    JE EXIT

    CMP AL,' '
    JNE SKIP1
    ADD SPACES,1
    JMP NEXT

SKIP1:
    CMP AL,'0'
    JL NEXT
    CMP AL,'9'
```

```

JG SKIP2
ADD DIGITS,1
JMP NEXT

SKIP2:
CMP AL,'A'
JL SKIP3
CMP AL,'Z'
JG SKIP3
CMP AL,'A'
JE VOWELS_INCR
CMP AL,'E'
JE VOWELS_INCR
CMP AL,'I'
JE VOWELS_INCR
CMP AL,'O'
JE VOWELS_INCR
CMP AL,'U'
JE VOWELS_INCR

ADD CONSONANTS,1
JMP NEXT

SKIP3:
CMP AL,'a'
JL NEXT
CMP AL,'z'
JG NEXT
CMP AL,'a'
JE VOWELS_INCR
CMP AL,'e'
JE VOWELS_INCR
CMP AL,'i'
JE VOWELS_INCR
CMP AL,'o'
JE VOWELS_INCR
CMP AL,'u'
JE VOWELS_INCR

ADD CONSONANTS,1
JMP NEXT

VOWELS_INCR:
ADD VOWELS,1

JMP NEXT

EXIT:

MOV AH,2
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

```

```
LEA DX,MSG2
MOV AH,9
INT 21H

MOV AH,2
MOV DL,VOWELS
ADD DL,30H
INT 21H
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

LEA DX,MSG3
MOV AH,9
INT 21H
MOV AH,2
MOV DL,CONSONANTS
ADD DL,30H
INT 21H
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

LEA DX,MSG4
MOV AH,9
INT 21H
MOV AH,2
MOV DL,DIGITS
ADD DL,30H
INT 21H
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

LEA DX,MSG5
MOV AH,9
INT 21H
MOV AH,2
MOV DL,SPACES
ADD DL,30H
INT 21H

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

Input:

Enter a string:A Quick Brown Fox 123 *#

Output:

Vowels:5

Consonants:9

Digits:3

Spaces:5

Title: String Searching Problem.**Outline:**

Write an assembly language program to implement a searching program that will read a string (a line of letters of English alphabet) as input and then find out the first capital letter which is occurred first and last capital letter which is occurred last in the alphabetical order as output. Your program will display a message with “No Capitals” when there is no capital letter in the string. Your program must contain necessary messages for formatted input and output.

Task: Implementation using Microsoft Macro-Assembler (MASM)**Mode of Evaluation: Experiment + Viva****Percentage of Weight: 10%****Algorithm**

Step-1: Begin

Step-2: Declare a SMALL memory MODEL

Step-3: Declare a 100H size STACK

Step-4: Declare an input message

Step-5: Declare an output message for First Capital

Step-6: Declare an output message for Last Capital

Step-7: Declare an output message for No Capital

Step-8: Declare a byte variable FIRST and set to [

Step-9: Declare a byte variable LAST and set to @

Step-10: Initialize Data Segment

Step-11: Display input message

Step-12: Move 1 to AH

Step-13: Call interrupt 21H

Step-14: Compare AL to 0DH

Step-15: If AL equal to 0DH Then go to Step-24

Step-14: Compare AL to 'A'

Step-15: If AL is less than 'A' Then go to Step-13

Step-16: Compare AL to 'Z'

Step-17: If AL is greater than 'Z' Then go to Step-13

Step-18: Compare AL to FIRST

Step-19: If AL is greater than equal to FIRST Then go to Step-21

Step-20: Move AL to FIRST

Step-21: Compare AL to LAST

Step-22: If AL is less than equal to LAST Then go to Step-13

Step-23: Move AL to LAST

Step-24: Compare FIRST to [

Step-25: If FIRST not equal to [Then go to Step-30

Step-26: Display First Capital Message

Step-27: Display the value of FIRST

Step-28: Display Last Capital Message

Step-29: Display the value of LAST

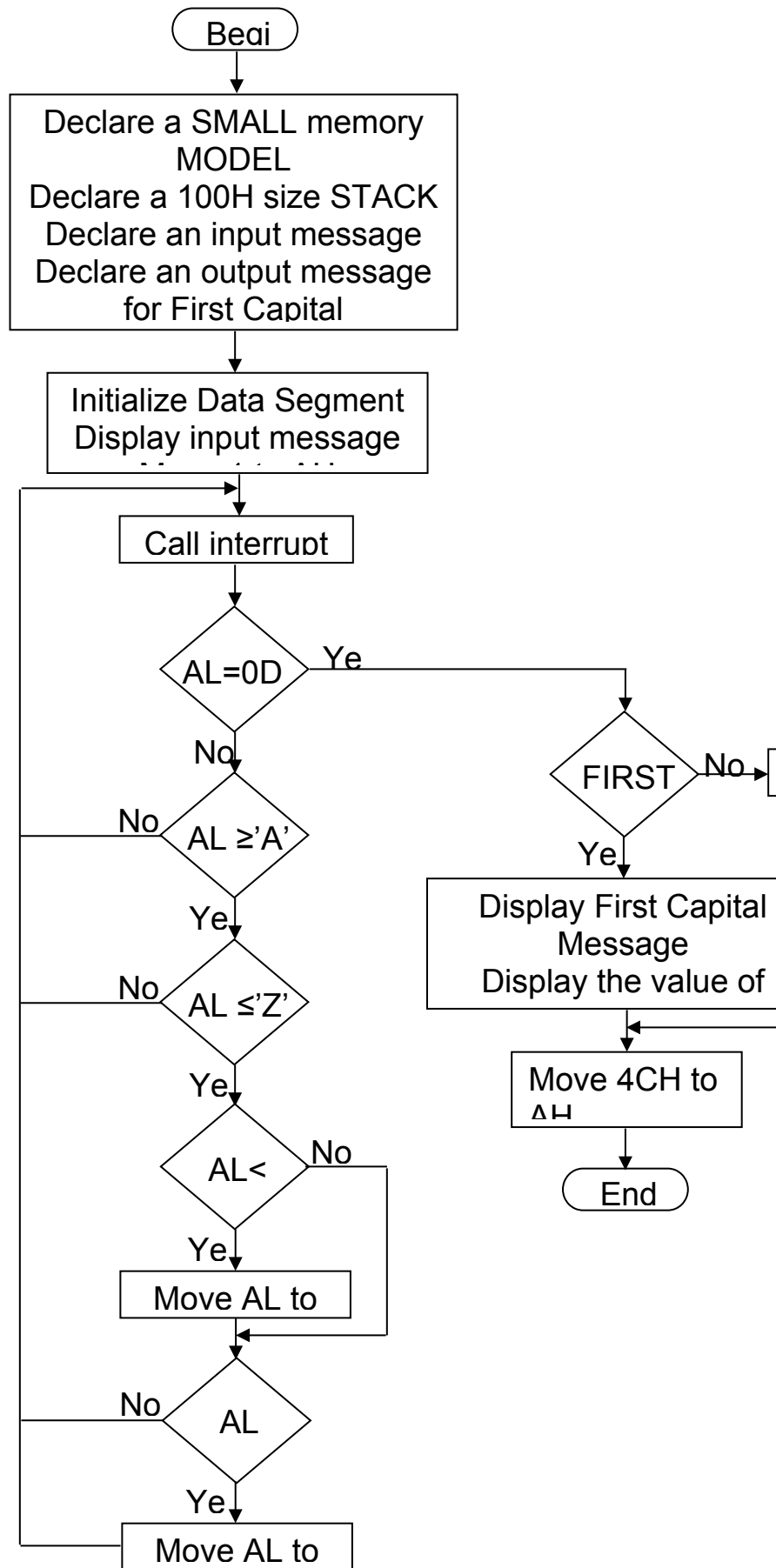
Step-30: Display No Capital Message

Step-31: Move 4CH to AH

Step-32: Call interrupt 21H

Step-33: End

Flowchart:



Source Code:

TITLE : A Program for Searching First Capital Letter and Last Capital Letter in a String

```
.MODEL SMALL
.STACK 100H

.DATA
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'Enter a string for First Capital and Last Capital Letter
Searching:',CR,LF, '$'
    MSG2 DB CR,LF,'The First Capital Letter is: $'
    MSG3 DB CR,LF,'The Last Capital Letter is: $'
    MSG4 DB CR,LF,'There is no Capital Letter in the String $'
    FIRST DB '['
    LAST DB '@'

.CODE

MAIN PROC
MOV AX,@DATA
MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1

NEXT:
    INT 21H
    CMP AL,0DH
    JE EXIT
    CMP AL,'A'
    JL NEXT
    CMP AL,'Z'
    JG NEXT
    CMP AL,FIRST
    JG SKIP2
    MOV FIRST,AL
SKIP2:
    CMP AL,LAST
    JL SKIP3
    MOV LAST,AL
SKIP3:
    JMP NEXT

EXIT:
    CMP FIRST,'['
    JNE SKIP4
    LEA DX,MSG4
    MOV AH,9
```

```
INT 21H
JMP EXIT2

SKIP4:
LEA DX,MSG2
MOV AH,9
INT 21H
MOV AH,2
MOV DL,FIRST
INT 21H

LEA DX,MSG3
MOV AH,9
INT 21H
MOV AH,2
MOV DL,LAST
INT 21H

EXIT2:
MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

Sample

Input:

Enter a string for First Capital and Last Capital Letter Searching:
The Quick Brown Fox Jumps Over the Lazy Dog

Output:

The First Capital Letter is:B
The Last Capital Letter is:Q

Number Processing Problems:

Title: Two Integer Numbers Adding Problem.

Outline:

Write an assembly language program to implement an adding program that will read two integer numbers of any length (up to 99 digits) and then add the two numbers. Finally, display the added value. Your program must contain necessary messages for formatted input and output.

Task: Implementation using Microsoft Macro-Assembler (MASM)

Mode of Evaluation: Experiment + Viva

Percentage of Weight: 10%

Algorithm

Step-1: Begin

Step-2: Declare a SMALL memory MODEL

Step-3: Declare a 100H size STACK

Step-4: Declare an input message for first number

Step-5: Declare an input message for second number

Step-6: Declare an output message

Step-8: Declare X as a 100 bytes array

Step-8: Declare Y as a 100 bytes array

Step-9: Declare Z as a 100 bytes array

Step-10: Initialize Data Segment

Step-11: Display input message for first number

Step-12: Move 0 to SI

Step-13: Move 1 to AH

Step-14: Call interrupt 21H

Step-15: Compare AL to 0DH

Step-16: If AL equal to 0DH Then go to Step-20

Step-17: Subtract 30H from AL

Step-18: Move AL to X[SI]

Step-19: Increment SI by 1

Step-20: Display input message for second number

Step-21: Move 0 to DI

Step-22: Move 1 to AH

Step-23: Call interrupt 21H

Step-24: Compare AL to 0DH

Step-25: If AL equal to 0DH Then go to Step-29

Step-26: Subtract 30H from AL

Step-27: Move AL to X[DI]

Step-28: Increment DI by 1

Step-29: Compare SI to DI

Step-30: If SI is greater than equal to DI Then go to Step-32

Step-31: Move DI to CX

Step-32: Move SI to CX

Step-33: Move 0 to AL

Step-34: Move 10 to BL

Step-35: Move 0 to BP

Step-36: Compare SI to 0

Step-37: If SI is greater than equal to 0 Then go to Step-40

Step-38: Compare DI to 0

Step-39: If DI is less than 0 Then go to Step-57

Step-40: Move 0 to DX

Step-41: Decrement SI by 1

Step-42: Compare SI to 0

Step-43: If SI is less than 0 Then go to Step-45

Step-44: Add DL to X[SI]

Step-45: Decrement DI by 1

Step-46: Compare DI to 0

Step-47: If SI is less than 0 Then go to Step-49

Step-48: Add DL to Y[DI]

Step-49: Add AL to DL

Step-50: Move DX to AX

Step-51: Divide AX by BL

Step-52: Move AH to Z[BP]

Step-53: Increment BP by 1

Step-54: Decrement CX by 1

Step-55: Compare CX to 0

Step-56: If CX is greater than 0 Then go to Step-36

Step-57: Compare AL to 0

Step-58: If AL equal to 0 Then go to Step-61

Step-59: Move AL to Z[BP]

Step-62: Go to Step-62

Step-61: Decrement BP by 1

Step-62: Display output message

Step-63: Move 2 to AH

Step-64: Move 0 to BX

Step-65: Move Z[BP] to DL

Step-66: Add 30H to DL

Step-67: Call interrupt 21H

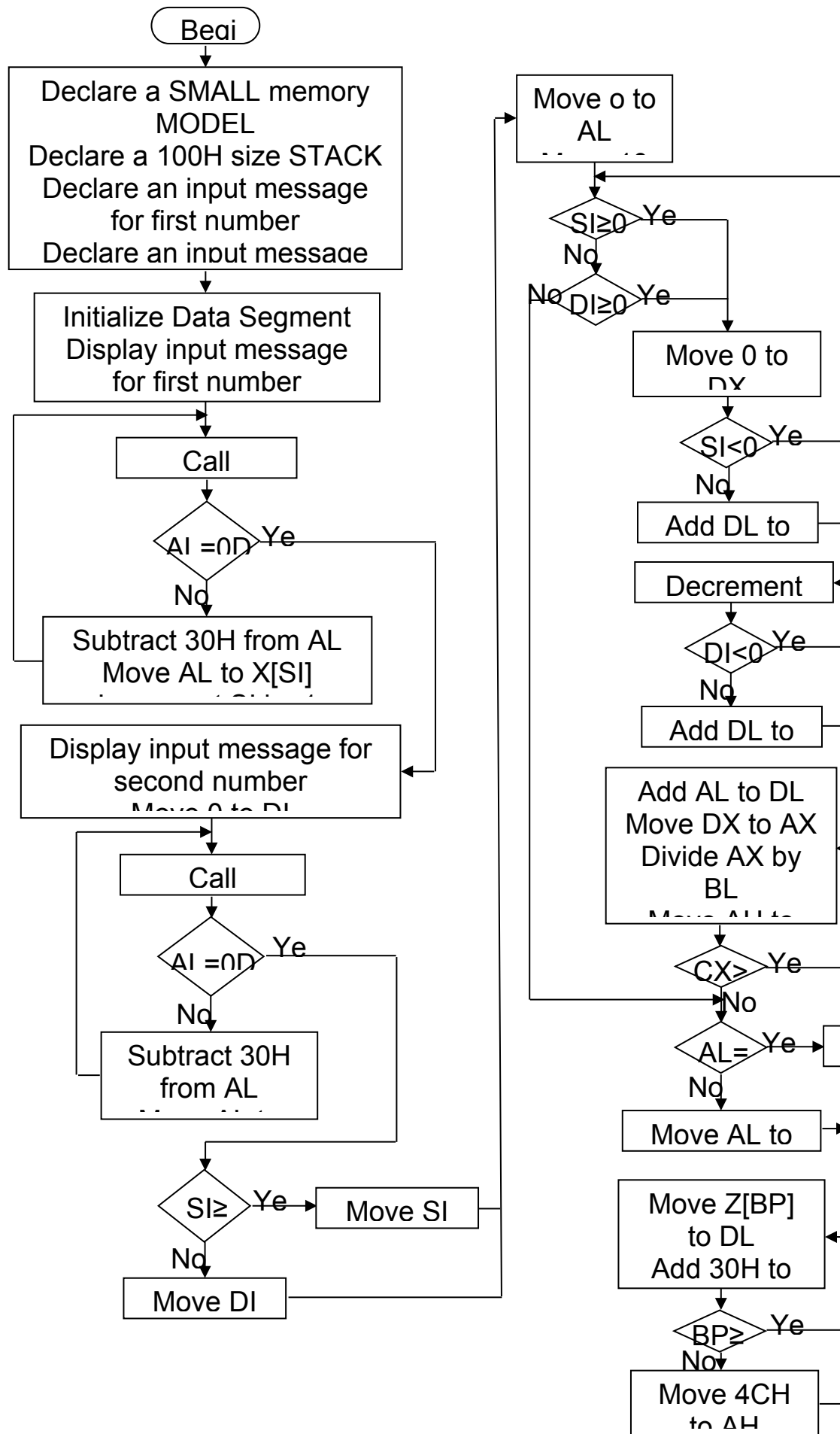
Step-68: Decrement BP by 1

Step-69: Compare BP to 0

Step-70: If BP is greater than equal to 0 Then go to Step-65

Step-71: Move 4CH to AH

Step-72: Call interrupt 21H

Flowchart:

Source Code:

TITLE : A Program for Adding two Integer Numbers of any Length (up to 99 digits)

```
.MODEL SMALL
.STACK 100H

.DATA
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'Enter first integer number:',CR,LF, '$'
    MSG2 DB CR,LF,'Enter second integer number:',CR,LF, '$'
    MSG3 DB CR,LF,'The sum is: $'
    X DB 100 DUP(?)
    Y DB 100 DUP(?)
    Z DB 100 DUP(?)

.CODE

MAIN PROC
MOV AX,@DATA
MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV SI,0
    MOV AH,1
NEXT1:
    INT 21H
    CMP AL,0DH
    JZ EXIT1
    SUB AL,30H
    MOV X[SI],AL
    INC SI
    JMP NEXT1

EXIT1:

    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV DI,0
    MOV AH,1

NEXT2:
    INT 21H
    CMP AL,0DH
    JZ EXIT2
    SUB AL,30H
    MOV Y[DI],AL
    INC DI
```


JMP NEXT2

EXIT2:

CMP SI,DI

JGE SKIP2

MOV CX,DI

JMP SKIP3

SKIP2:

MOV CX,SI

SKIP3:

MOV AL,0

MOV BL,10

MOV BP,0

NEXT3:

CMP SI,0

JGE SKIP6

CMP DI,0

JGE SKIP6

JMP EXIT3

SKIP6:

MOV DX,0

DEC SI

JL SKIP4

ADD DL,X[SI]

SKIP4:

DEC DI

JL SKIP5

ADD DL,Y[DI]

SKIP5:

ADD DL,AL

MOV AX,DX

DIV BL

MOV Z[BP],AH

INC BP

DEC CX

JG NEXT3

EXIT3:

CMP AL,0

JE SKIP7

MOV Z[BP],AL

JMP SKIP8

SKIP7:

DEC BP

SKIP8:

LEA DX,MSG3

MOV AH,9

INT 21H

MOV AH,2

MOV BX,0

```
NEXT4:
MOV DL,Z[BP]
ADD DL,30H
INT 21H
DEC BP
JGE NEXT4

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

Sample

Input:

Enter first integer number:

2345

Enter second integer number:

6789

Output:

The sum is:9134
